Name : Youssef mohamed yasser

ID: 2305038

Buffer Overflow Assignment Report

1. Introduction

This report discusses the concept of buffer overflow vulnerabilities, their impact on software systems, and the results of a controlled laboratory exercise conducted to observe overflow behavior in a safe environment.

2. Objective

The objective of this experiment is to understand how improper handling of input can lead to memory corruption. The assignment focuses on analyzing a vulnerable program, observing its behavior under different input sizes, and documenting the effects without executing harmful actions.

3. Environment Setup

- Virtual Machine (VM) running a Linux distribution
- Compiler and debugger tools installed
- All testing performed in an isolated and safe environment

4. Source Code Review

The reviewed program contains a statically sized buffer that does not enforce input length checks. This allows user input to exceed the allocated buffer, leading to memory overwriting. The issue arises from unsafe input functions that do not validate boundaries.

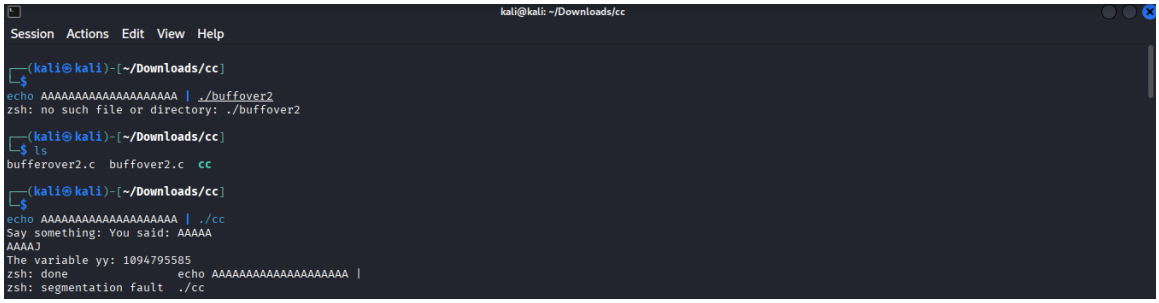
5. Methodology

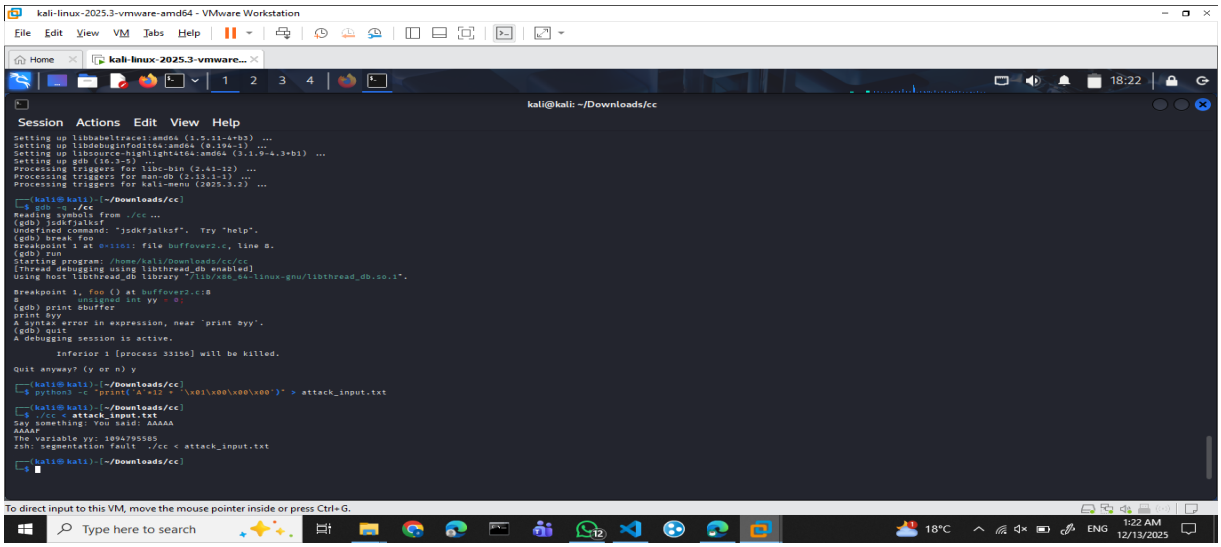
1. Compile the program in a controlled environment.
2. Execute the program with normal inputs and record expected behavior.
3. Gradually increase input size and observe changes.
4. Use a debugger to inspect stack variables before and after large inputs.
5. Document observations.

6. Observations

During the experiment, normal-sized inputs produced stable and expected output. Larger inputs resulted in irregular variable values and abnormal program termination. Debugger

inspection confirmed that excessive input data overwrote memory regions adjacent to the buffer.





Source code for the attacks:

1. // buffover2.c

```
#include <stdio.h>

int foo();

int main() {

    while (1) foo();

    return 0;

}

int foo(){

    unsigned int yy = 0;
```

```
    char buffer [5]; char ch; int i = 0;

    printf("Say something: ");

    while ((ch = getchar()) != '\n') buffer [i++] = ch;

    buffer[i] = '\0';

    printf("You said: %s\n", buffer);

    printf("The variable yy: %d\n", yy);

    return 0;

}
```

2. // buffover2.c

```
#include <stdio.h>



int foo();



int main() {

    while (1) foo();
```

```
    return 0;
}


int foo(){
    unsigned int yy = 0;
    char buffer [5];
    char ch;
    int i=0;


    printf("Say something: ");


    // حلقة القراءة التي لا تتحقق من حدود المخزن، مما يسمح بالتجاوز
    while ((ch= getchar()) != '\n')
        buffer [i++]=ch;
```

```
    buffer[i] = '\0';

    printf("You said: %s\n", buffer);

    printf("The variable yy: %d\n", yy);



    return 0;

}
```

7. Analysis

The experiment demonstrates that unchecked input leads to stack corruption. This occurs because the buffer resides near critical values on the stack, including local variables and saved frame pointers. When the buffer is overrun, the excess data overwrites these regions, causing unexpected behavior.

8. Preventive Measures

To prevent buffer overflow vulnerabilities, developers should:

- Use input functions that enforce size limits
- Apply compiler-level protections such as stack canaries
- Enable operating system features such as ASLR and NX bit
- Conduct code reviews and static analysis

9. Conclusion

This controlled exercise highlights the risk of buffer overflow vulnerabilities and the importance of safe programming practices. Understanding how memory corruption occurs is essential for developing secure systems and avoiding critical security flaws.